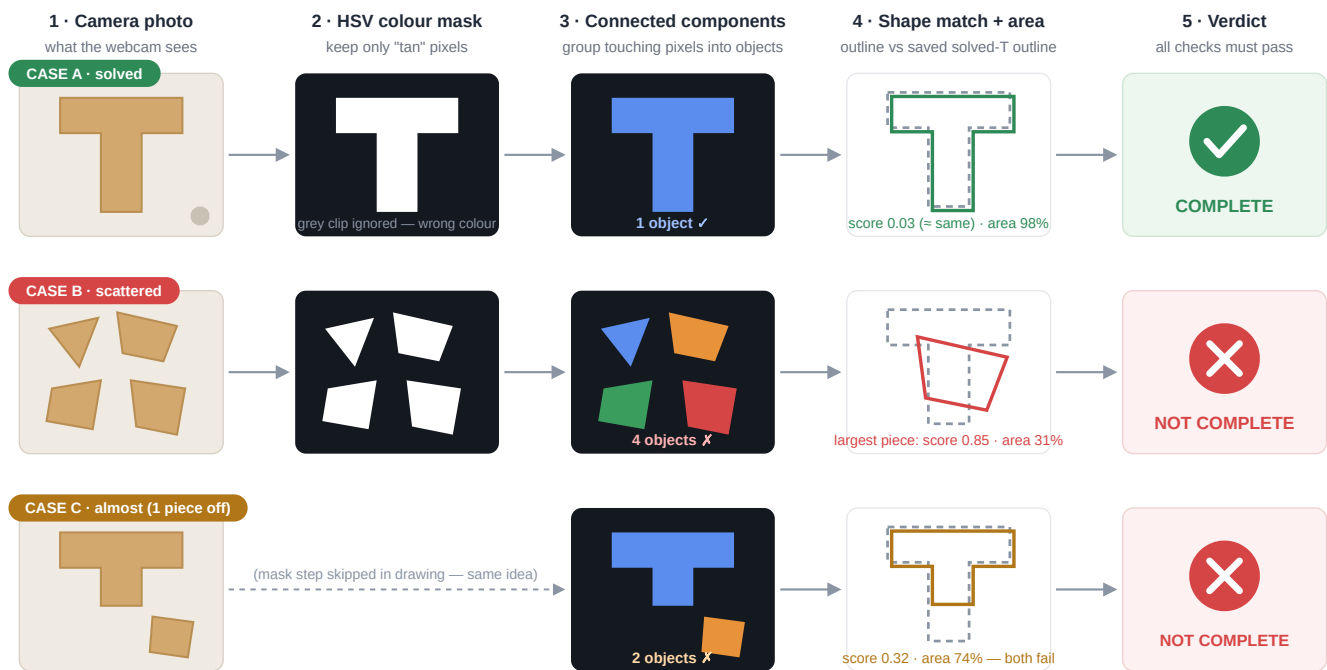


How the T-puzzle completion check works

Classical computer vision — rules, not machine learning · runs on the live camera frame at every switch press (start check + stop checks) · implemented in `src/vision/verify.py` · Param Foundation, AI & Robotics Lab · 20 Aug 2026

The same 4 steps, two different outcomes



Colour only FINDS the cardboard (step 2). The COMPLETE/NOT decision comes from steps 3–4: one object, right outline, right size.

Decision rule: COMPLETE = exactly 1 tan object AND shape-match score ≤ 0.08 AND area between 82–118% of the saved solved-T
(these are the live values in `config.yaml` → completion: — calibrate against real labelled photos)

Each step, one level deeper

2 · HSV colour mask — `cv2.inRange()`

The photo is converted from RGB to HSV, where colour (Hue) separates from brightness (Value) — so shadows hurt much less. Every pixel inside the "tan" range turns white, everything else black.



3 · Connected components —

`cv2.connectedComponents()`

White pixels that touch each other are grouped into one labelled object. This is the key insight: a **solved** puzzle has all pieces touching → **one** object. Scattered or half-done → several objects. Tiny specks below a minimum area (dust, glare) are thrown away first.

4a · Contour matching —

`cv2.matchShapes()`

The outline (`cv2.findContours()`) of the single object is compared with the outline of a reference photo of the solved T, taken once during setup. The score uses Hu moments, which don't care about position or rotation — a T anywhere on the board still matches. **0** = **identical shape**; bigger = more different.

4b · Area-ratio check —

`cv2.contourArea()`

Safety net for shape-match false positives: the object's pixel area must be roughly the reference T's area (camera is fixed, so scale is constant). A single piece that happens to look T-ish still fails, because it's far too small.

When it runs + why it's reliable here

It runs on the **live camera frame at each switch press** — once to validate the start (already solved? → press ignored) and at every stop attempt. Nothing is stored per session; only the solved reference image + mask are kept on disk. Fixed top-down camera, fixed board, known puzzle — the conditions where rule-based vision beats ML: no training data, every decision explainable (within its thresholds — it matches the reference shape, it does not truly "recognise" a T).

Tuning knobs (already set in `config.yaml` → `completion:`)

Knob	Current value	Adjust when...
HSV tan range	H 5–35, S 45–255, V 70–235	mask misses pieces / grabs the board
Min blob size	2000 px	noise blobs appear / small pieces vanish
Mask clean-up kernel	5 px morphology	speckles or pinholes in the mask
Match threshold	score ≤ 0.08	false rejects / false accepts
Area band	82–118%	lighting changes apparent size